

Task 5

In the previous tasks, the robot was controlled using a differential drive mechanism, where the motion of the robot was generated using two independently driven wheels. Although differential drive robots are simple and widely used, they cannot move in every direction without first changing their orientation. For example, to move sideways, a differential drive robot must first rotate and then move forward.

A **holonomic robot** overcomes this limitation. It is capable of moving independently along the X-axis, Y-axis, and rotating about its vertical axis simultaneously. This means the robot can translate in any direction while maintaining its orientation, making it highly suitable for applications requiring high maneuverability, such as warehouse automation, robot soccer, and service robots.

The robot used in this task is a **three-wheeled omnidirectional (holonomic) robot**. It consists of three omni wheels placed **120° apart** around the robot chassis. Each wheel is driven independently by a motor.

Unlike conventional wheels, an **omni wheel** consists of a primary wheel with several passive rollers mounted around its circumference. The wheel generates traction only along its rolling direction, while the passive rollers allow free motion perpendicular to that direction. By combining the motion of all three wheels, the robot can move in any planar direction.



The motion of a holonomic robot is represented using three independent body velocities:

- v_x : Linear velocity along the robot's X-axis.
- v_y : Linear velocity along the robot's Y-axis.
- ω : Angular velocity about the robot's Z-axis.

Instructions

- The robot URDF/Xacro files and controller.yaml are already provided in the mini_bot package. No modifications are required unless explicitly mentioned.

- The Gazebo world, arena models, overhead camera, and sensor_bridge.yaml are already provided in the task_arena package.
- All your implementations for these tasks must be written only inside the bot_control package. Do not modify any other package unless instructed.
- After creating a new Python node, add the corresponding entry point in setup.py before building the workspace.
- Rebuild and source the workspace after every change.
- Use `ros_task_2.launch.py` to launch gazebo and `display.launch.py` to view the bot in RViz.

Pull the changes from [GitHub](#) .

TASK 5A: Kinematics of a Three-Wheeled Holonomic Robot

Objective

Implement the **Inverse Kinematics (IK)** and **Forward Kinematics (FK)** of a three-wheeled holonomic robot.

Background

The wheel velocity controller accepts the angular velocity of each wheel as its input. Therefore, to achieve a desired robot motion, the required wheel angular velocities must first be calculated using **Inverse Kinematics** using the bot's linear velocity and angular velocity.

Conversely, the robot estimates its motion from the measured wheel angular velocities.

Forward Kinematics converts the wheel angular velocities back into the robot's linear and angular velocities. You have to calculate using angular velocity of wheels because in real hardware the given velocities and received velocities at the wheels may not be same because of hardware limitation.

Steps

1. Derive the Inverse Kinematics equations for the three-wheeled holonomic robot.
2. Derive the Forward Kinematics equations corresponding to the derived IK equations.
3. Implement both your logic in **ik.py** and **fk.py** inside the bot_control packages respectively.

Note:

The `ik.py` and `fk.py` files should only contain the implementation of the corresponding functions. Do **not** define fixed values of v_x , v_y , ω , or wheel angular velocities inside these files. The required inputs will be provided by another ROS 2 node, which will call these functions and use their outputs accordingly.

Odometry

Background

In Gazebo, the robot's pose can be obtained directly from the simulator, allowing us to know its exact position and orientation in the world. However, in a real robot, there is no system that directly provides the robot's actual coordinates. Therefore, the robot estimates its own position using **Odometry**.

The **odom** frame is initialized at the robot's starting position and serves as a continuously updated reference frame. The robot estimates its motion using measurements from the wheels. In real hardware, wheel encoders measure the wheel rotation, which is converted into wheel angular velocities. These angular velocities are then used in the **Forward Kinematics** equations to estimate the robot's linear and angular velocities (v_x, v_y, ω). By integrating these velocities over time, the robot estimates its position and orientation.

The estimated pose is published on the **/odom** topic, and the corresponding **odom** → **footprint_link** transform is broadcast, allowing other ROS nodes to use the robot's estimated position for navigation and control.

You are instructed to see the implementation of odometry calculation in the file named **odom_broadcaster.py**. During PI, questions will be asked on its derivation and the theory of Odometry and /odom topic.

TASK 5B: Trajectory Generation using Inverse Kinematics

Objective

The objective of this task is to use the previously implemented **Inverse Kinematics (IK)** and **Forward Kinematics (FK)** along with **Odometry** to make the robot trace an **elliptical trajectory** and an **Archimedean spiral**. Unlike the previous approach, the robot must continuously use its estimated position from the **/odom** topic as feedback to correct its motion while tracing the desired trajectory.

Background

In the previous task, the **Inverse Kinematics (IK)** and **Forward Kinematics (FK)** of the robot were implemented. The provided **Odometry Broadcaster** uses the measured wheel angular velocities together with your **Forward Kinematics** implementation to estimate the robot's position and orientation. The estimated pose is published on the **/odom** topic.

In this task, instead of commanding fixed robot velocities (open-loop control), you will implement a **closed-loop controller**. A desired point is continuously generated on the required trajectory (ellipse or spiral), and the robot's current position obtained from the **/odom** topic is used to compute the position error. Based on this error, the required robot velocity is calculated, converted into wheel angular velocities using your **inverse_kinematics()** function, and

published to the wheel velocity controller. By continuously correcting its motion using odometry feedback, the robot is able to follow the desired trajectory.

Two trajectory plotting nodes are provided in the `bot_control` package:

- `trace_path_odom.py` plots the robot trajectory using the estimated pose from the `/odom` topic.
- `trace_ground_truth.py` plots the robot's actual trajectory using Gazebo's ground truth pose.

Steps

1. Use the provided `odom_publisher.py`, `trace_path_odom.py`, and `trace_ground_truth.py` without making any modifications.
2. Create two ROS 2 Python nodes in the `bot_control` package:
 - `ellipse.py`
 - `spiral.py`
3. Each node should:
 - Subscribe to the `/odom` topic to obtain the robot's current position and orientation.
 - Generate the desired trajectory (ellipse or spiral).
 - Continuously compute the position error between the robot and the desired trajectory.
 - Calculate the required robot linear velocities using the position error.
 - Use the `inverse_kinematics()` function implemented in `ik.py` to compute the wheel angular velocities.
 - Publish the computed wheel angular velocities to `/wheel_velocity_controller/commands` using a `Float64MultiArray` message.
4. Launch the simulation. The odometry broadcaster will automatically start with the launch file. Verify that the `/odom` topic is available using: `ros2 topic list`.
5. Run both trajectory plotting nodes in separate terminals. Execute your ellipse or spiral node. Once you feel the trajectory has been completed, stop the tracing nodes using `Ctrl+C` to visualize the generated trajectories.

Submission Instructions

Upload the following files in the TASK5B directory:

- Both the files having implementation of `ellipse.py` and `spiral.py`
- A screen recording demonstrating:
 - The robot tracing the trajectories in Gazebo.
 - The corresponding trajectories generated using the provided odometry plotting node and gazebo pose plotting node.

Note: The `odom_publisher.py` and trajectory plotting nodes will be provided. Do not modify these files. Your implementation should use the `ik.py` and `fk.py` files developed in **TASK 5A**.

Observation

Observe the trajectory generated using the provided odometry plotting node.

- If the trajectory obtained from the `trace_path_odom` matches the expected trajectory (ellipse/spiral), it indicates that your Inverse Kinematics (IK) and Forward Kinematics (FK) implementations are functioning correctly.
- If it deviates significantly from the expected path, revisit your IK and FK derivations and implementation, as errors in these calculations will directly affect the estimated robot motion.
- In general, the trajectory generated using `trace_path_odom` closely follows the expected trajectory because it is generated in the odom frame, i.e., it represents the robot's estimated motion based on its own odometry. However, the trajectory generated using `trace_ground_truth` may differ from the odometry trajectory because it represents the robot's actual motion in the Gazebo world.

Trade-off of Using Odometry

Odometry allows the robot to estimate its own position without relying on any external coordinate system or localization system. This makes odometry an essential tool for local motion estimation, path tracking, and localization.

The drawback of odometry is that the estimated pose accumulates error over time due to wheel slip, imperfect lateral motion, friction, contact dynamics, encoder inaccuracies, and other effects not captured by the ideal kinematic model. Therefore, the trajectory generated using `trace_path_odom` generally follows the expected path, while the `trace_ground_truth` trajectory in Gazebo may differ. Although odometry is well suited for local localization, it is not sufficient for accurate global localization. In practice, this drift is reduced using techniques such as closed-loop feedback control, LiDAR/Visual SLAM, IMUs, and sensor fusion algorithms like the Extended Kalman Filter (EKF).

TASK 5C: ArUco detection using an Overhead Camera

Objective

The objective of this task is to localize the robot using an overhead camera and ArUco markers. Students will detect all the ArUco markers present in the arena, estimate the robot's position and orientation using its ArUco marker, and publish the robot pose on a custom topic named `/bot_pose`. This task introduces vision-based localization as an alternative to wheel-encoder-based odometry.

Background

In the previous tasks, the robot pose was estimated using wheel encoder measurements and Forward Kinematics, and published on the `/odom` topic. In this task, localization is performed using an overhead camera. The camera continuously observes the robot and estimates its pose using the ArUco marker mounted on the robot. Unlike `/odom`, the robot pose in this task is represented in the camera image frame i.e. in pixel coordinates.

Steps

1. Subscribe to the overhead camera: Create a ROS 2 Python node named `feedback.py`.
Subscribe to the following topic: `/camera`
2. Detect ArUco markers Using OpenCV:
 - Detect all ArUco markers in the arena.
 - Extract the pixel coordinates of every detected marker.
 - Store the centre coordinates of the corner markers for later use.
 - Detect the robot's ArUco marker using its unique ID.
3. Estimate robot pose: Using the detected robot marker:
 - Calculate the robot centre in image coordinates.
 - Estimate the robot orientation (θ).
 - Publish the robot pose on `/bot_pose` using `geometry_msgs/msg/Pose2D` where $x \rightarrow$ image x-coordinate (pixels), $y \rightarrow$ image y-coordinate (pixels), and $\theta \rightarrow$ robot orientation (radians).
4. Visualisation: Display the camera feed with detected ArUco markers, marker IDs, robot heading, and robot centre.

Submission

Upload:

- `feedback.py`
- A screen recording demonstrating ArUco detection with ArUco IDs displayed, robot pose estimation, and publishing `/bot_pose`.

TASK 5D: Vision-Based Closed-Loop Go-to-Goal Navigation

Objective

The objective of this task is to implement a closed-loop Go-to-Goal controller using the vision-based robot pose obtained from Task 5C. Unlike Task 5B, where the robot pose was obtained from wheel odometry (`/odom`), this task uses the robot pose estimated from the overhead camera (`/bot_pose`) to navigate the robot to the corner ArUco markers. The primary goal of this task is to demonstrate that the control algorithm remains unchanged, while only the source of localization changes.

Background

In Task 5B, the controller obtained the robot pose from /odom which was generated using Forward Kinematics and wheel velocity measurements. In this task, the controller obtains the robot pose from /bot_pose which is generated using ArUco marker detection from the overhead camera.

Steps

1. **Create the Control Node:** Create a ROS 2 Python node named `go_to_goal.py` inside the `bot_control` package.
2. **Subscribe to Robot Pose:** Subscribe to `/bot_pose` which publishes the robot pose as a `geometry_msgs/msg/Pose2D` message. Extract the robot x-coordinate, y-coordinate, and orientation (θ) from the received message.
3. **Define Goal Points:** Use the corner marker coordinates obtained in Task 5C as the navigation goals. The robot should visit the corner markers in the following order: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2$. Store these coordinates in an appropriate data structure.
4. **Implement Closed-Loop Go-to-Goal Control:** Continuously compare the robot's current position with the active goal point. Compute the position error between the robot and the current goal. Using a proportional controller, generate the required robot velocity (v_x , v_y , ω) required to reduce the position error.
5. **Reuse the Inverse Kinematics:** Use the `inverse_kinematics()` function developed in Task 5A to convert the desired robot velocities into wheel angular velocities. Publish the computed wheel velocities to `/wheel_velocity_controller/commands` using a `Float64MultiArray` message.
6. **Goal Switching:** Once the robot reaches the current goal within a predefined threshold, switch to the next goal, and continue until all corner markers have been visited. After visiting the final goal, stop the robot by publishing zero wheel velocities.
7. Run the `visualization.py` to track the live motion of the bot.

Submission Instructions

Upload the following files in the TASK5D directory:

- `go_to_goal.py`
- Upload a screen recording (`TASK5D.mp4`) demonstrating the robot navigating between all four corner ArUco markers, closed-loop correction of the robot's motion using the overhead camera, and the trajectory visualized using the provided visualization node.

DEADLINE 12 July EOD

You are required to update your github with this last task on or before the said deadline. Tasks updated after the deadline won't be considered for evaluation. If time permits your interview will be conducted before the college reopens.